JavaScript Runtime & TypeScript

Ce cours pose les bases théoriques du runtime JavaScript (mono-thread, concurrence, communication temps réel) puis le système de types de TypeScript, du socle strict aux patterns avancés.

I. Le Runtime JavaScript : Anatomie du Single Thread

Le mythe du multi-thread en JavaScript

JavaScript est un langage **mono-thread** par conception. Son moteur d'exécution (V8 dans Chrome et Node.js, JavaScriptCore dans Safari, SpiderMonkey dans Firefox) exécute le code d'un contexte donné sur **une seule pile d'appels** (*Call Stack*), avec un seul tas de mémoire (*Memory Heap*).

Une seule instruction s'exécute à un instant `t` sur le thread principal (*Main Thread*).

Pour ne pas bloquer l'interface ou le serveur lors d'opérations lentes (requêtes réseau, lecture disque, minuteurs), le runtime délègue ces tâches à l'environnement hôte (Browser Web APIs ou libuv sous Node.js) et orchestre leur résolution avec l'**Event Loop**.

Doc officielle (Spécification WHATWG HTML) : https://html.spec.whatwg.org/multipage/webappapis.html#event-loops

La Call Stack et le Stack Overflow

La *Call Stack* suit la règle LIFO (*Last In, First Out*). Chaque appel de fonction empile une *stack frame* avec ses arguments et ses variables locales. Le dernier cadre empilé est toujours le premier dépilé.

Si des appels récursifs s'enchaînent sans condition d'arrêt, la pile sature la limite de mémoire allouée. Le moteur lève alors `RangeError: Maximum call stack size exceeded` .

La Call Stack et le Stack Overflow (suite)

Un appel imbriqué empile ses frames dans l'ordre de l'appel :

```
function processOrder(user) {
  return validateUser(user);
}

function validateUser(user) {
  return formatUserName(user);
}

function formatUserName(user) {
  if (!user.lastName) {
    throw new Error("lastName manquant");
  }
  return `${user.firstName} ${user.lastName}`;
}

processOrder({ firstName: "Alice" });
```

La Call Stack et le Stack Overflow (fin)

La pile contient dans l'ordre `processOrder`, puis `validateUser`, puis `formatUserName`. Quand l'erreur est levée dans `formatUserName`, la stack trace liste les appels du plus récent au plus ancien, exactement dans l'ordre inverse de l'empilement :

```
Uncaught Error: lastName manquant
    at formatUserName
    at validateUser
    at processOrder
```

Lire une stack trace, c'est lire la Call Stack au moment du crash. Le premier appel affiché est toujours celui où l'erreur a été levée, pas le point d'entrée du programme.

Doc MDN (Call Stack) : https://developer.mozilla.org/fr/docs/Glossary/Call_stack

L'Event Loop : anatomie d'un cycle d'exécution

L'Event Loop est une boucle infinie qui synchronise l'exécution du code. Un tour, dans un navigateur, tient en **trois étapes** :

1. **Task** : extraire **une seule tâche**, la plus ancienne exécutable dans l'une des files de tâches (*task queues*), puis l'exécuter. Une tâche est **un appel de fonction**. Elle démarre sur une pile vide et se termine quand cette pile est redevenue vide.
2. **Microtasks Queue** : dès que la Call Stack est vide, vider **immédiatement et totalement** la file des microtâches, y compris celles ajoutées pendant le vidage.
3. **Rendu graphique** : le navigateur détermine s'il a une **opportunité de rendu** (styles, layout, paint). Elle dépend de la fréquence de rafraîchissement de l'écran, de la visibilité du document et de la charge en cours, pas d'une estimation du coût du rendu. Sur un écran 60 Hz, une opportunité se présente environ toutes les 16,7 ms. L'étape est donc sautée à la plupart des tours.

Puis la boucle repart à l'étape 1.

L'Event Loop : le budget d'un tour

Ces 16,7 ms constituent le budget d'un tour complet, tâche et microtâches comprises. Le navigateur ne vérifie jamais ce budget avant de peindre. Il ne peut pas connaître le coût d'un layout avant de l'avoir calculé.

Quand le tour dépasse ce budget, rien n'est annulé. Le rendu va jusqu'au bout, la frame arrive en retard, le rafraîchissement suivant est manqué et la cadence tombe à 30 images par seconde.

Ce que « pile vide » veut dire

La Call Stack n'est pas remplie par la task queue. Une tâche n'est pas un morceau de pile, c'est **un seul appel de fonction**. La pile ne se vide donc pas en extrayant d'autres tâches, elle se vide **par l'intérieur**, à mesure que les fonctions appelées retournent :

```
setTimeout(handleClick, 0); // la tâche, c'est l'appel de handleClick

function handleClick() {
  validate();
  render();
}
```

Ce que « pile vide » veut dire (suite)

MOMENT	CALL STACK
La boucle extrait la tâche et appelle la fonction	`[handleClick]`
`handleClick` appelle `validate`	`[handleClick, validate]`
`validate` retourne	`[handleClick]`
`handleClick` appelle `render`	`[handleClick, render]`
`render` retourne, puis `handleClick` retourne	`[]`
Pile vide -> **la tâche est terminée**	`[]`

La pile vide, un signal, pas un objectif

La pile vide n'est pas un objectif que la boucle poursuivrait en dépilant des tâches. C'est le **signal** que la tâche courante est finie. MDN le formule ainsi : « A job is considered completed when **the stack is empty**; then, the next job is pulled from the queue. » Extraire une deuxième tâche ne viderait pas la pile, cela la remplirait de nouveau.

C'est aussi la définition du *run-to-completion* : « whenever a function runs, it cannot be preempted and will run entirely before any other code runs ». Une tâche démarrée va jusqu'au bout, sans interruption possible. D'où le gel de l'interface quand elle est longue.

Pourquoi une seule tâche, mais toutes les microtâches

Les deux files ne jouent pas le même rôle. Une microtâche n'est pas « la tâche suivante », c'est **la fin de la tâche courante**. Son traitement fait partie du travail de terminaison, d'où le nom de *microtask checkpoint* donné à l'étape 2.

- **Vider toute la file des microtâches** garantit qu'une chaîne de Promesses ne sera jamais coupée en deux par un rendu ou par un timer. C'est la **cohérence**.
- **N'extraire qu'une seule tâche** garantit qu'on atteint l'étape de rendu entre deux tâches. Si la boucle vidait la task queue entière, un flux continu de `setTimeout`, d'événements de scroll et de callbacks réseau repousserait le rendu indéfiniment. C'est la **réactivité**.

Pourquoi une seule tâche, mais toutes les microtâches (suite)

Cette première garantie a un prix. Une microtâche qui se réinjecte sans fin l'exploite pour ne jamais rendre la main (*starvation*).

Note de vocabulaire : la page MDN ci-dessous emploie « job » / *job queue*, termes issus de la spécification ECMAScript, en précisant « or, in HTML terminology, an event loop ». La spec HTML dit « task » / *task queue*. Même objet, deux spécifications.

Doc MDN (Modèle d'exécution JavaScript) : https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Execution_model

Tasks vs Microtasks : priorités et ordonnancement

Une fonction asynchrone ne s'exécute jamais au moment où elle est déclenchée. Son callback est déposé dans une file d'attente, et attend son tour. Le runtime maintient deux files distinctes, avec des règles de vidage différentes.

Une **task** (*tâche*) est un callback ordinaire, planifié pour un futur tour de la boucle d'événements, sans urgence particulière. C'est l'unité que l'étape 1 extrait, **une seule par tour**.

Une **microtask** est un callback qui doit s'exécuter dès que possible, sans attendre le tour suivant. La file entière est vidée à l'étape 2, juste après la tâche courante.

Une mise en garde sur le mot « macrotask »

Le terme circule beaucoup, y compris dans des articles de référence, mais il **n'est pas normatif**.

Pourquoi deux files

Cette distinction vient directement de la sémantique des Promesses. Quand une Promesse se résout, son `.then()` doit s'exécuter au plus tôt pour que les chaînes de Promesses se comportent de façon prévisible, sans laisser un rendu graphique ou un `setTimeout` s'intercaler au milieu d'un enchaînement logique. Les Promesses utilisent donc la file des microtasks.

Toutes les fonctions de rappel asynchrones (*callbacks*) ne sont pas traitées avec la même priorité :

- **Microtasks** (priorité absolue) : Promesses (`Promise.prototype.then()` , `catch()` , `finally()`), `queueMicrotask()` , et `MutationObserver` .
- **Tasks** (callbacks ordinaires) : `setTimeout` , `setInterval` , événements DOM (clic, scroll), callbacks d'I/O réseau, l'exécution d'un `<script>` , et `setImmediate` côté Node.js uniquement.

Tasks vs Microtasks : l'ordre d'exécution

```
console.log("Synchrone");

setTimeout(() => console.log("Task (setTimeout)"), 0);

Promise.resolve().then(() => console.log("Microtask (Promise)"));

console.log("Synchrone");
// Ordre d'exécution : 1. Synchrone -> 2. Synchrone -> 3. Microtask -> 4. Task
```

Les deux `console.log("Synchrone")` **appartiennent déjà à une task**, celle qui exécute le script lui-même. Ils s'exécutent d'affilée parce qu'une task va toujours jusqu'au bout (*run-to-completion*), pas parce qu'une phase « synchrone » précéderait la boucle.

Doc MDN (Microtask Guide) : https://developer.mozilla.org/fr/docs/Web/API/HTML_DOM_API/Microtask_guide
Doc MDN (In depth: Microtasks and the JavaScript runtime environment) :
https://developer.mozilla.org/fr/docs/Web/API/HTML_DOM_API/Microtask_guide/In_depth

Le risque : le phénomène de starvation

L'Event Loop vide la file des microtâches en entier. Si une microtâche en cours ajoute une nouvelle microtâche à la file, celle-ci est traitée avant que l'Event Loop ne passe à l'étape 3 (rendu) ou n'entame le tour suivant. La file n'est vide que lorsqu'aucune microtâche n'attend plus, y compris celles ajoutées pendant le vidage.

Un callback qui réinjecte systématiquement une nouvelle microtâche piège donc l'Event Loop dans l'étape 2. Le rendu graphique et les tâches suivantes n'obtiennent jamais la main, puisque la condition de sortie de cette étape n'est jamais remplie. C'est la *starvation* :

```
function freezeEverything() {
  Promise.resolve().then(freezeEverything); // Vide la pile mais réinjecte une microtâche sans fin
}

freezeEverything(); // À ne tenter que dans un onglet jetable.
```

Le risque : starvation et INP

Ce phénomène empêche le navigateur d'afficher la moindre frame ou de répondre aux clics. Il provoque un gel complet de la page et une dégradation critique du score **INP** (Interaction to Next Paint).

Guide Web.dev (Optimisation des tâches longues & INP) : https://web.dev/articles/optimize-long-tasks

II. Parallélisme Réel & Concurrence

Concurrence asynchrone vs Parallélisme physique

- **Concurrence asynchrone (I/O non-bloquant)** : un seul thread jongle entre des tâches en attente (requête fetch, minuterie). Aucun calcul n'est accéléré.
- **Parallélisme réel (calculs CPU-intensifs)** : exécution physique simultanée sur plusieurs cœurs de processeur distincts.

Pour traiter des opérations lourdes (compression, parsing de flux volumineux, filtres d'image, cryptographie) sans geler l'UI, le code doit être délégué à des threads dédiés.

Doc Web.dev (Main thread work) : https://web.dev/articles/main-thread-work

Web Workers (Navigateur) & Worker Threads (Node.js)

Un Worker s'exécute dans un environnement d'exécution totalement isolé :

- Aucun accès direct au DOM, ni à l'objet `window` ou `document`.
- Sa propre Call Stack, sa propre mémoire et sa propre Event Loop.

```
// main.js
const worker = new Worker(new URL("./compute.worker.js", import.meta.url), {
  type: "module",
});

worker.postMessage({ items: [1, 2, 3, 4] });

worker.onmessage = (event) => {
  console.log("Résultat calculé en parallèle :", event.data);
};
```

Web Workers (Navigateur) & Worker Threads (Node.js) (suite)

```
// compute.worker.js
self.onmessage = (event) => {
  const result = event.data.items.map((x) => x ** 2);
  self.postMessage(result);
};
```

Doc MDN (Using Web Workers) : https://developer.mozilla.org/fr/docs/Web/API/Web_Workers_API/Using_web_workers

Doc Node.js (Worker Threads) : https://nodejs.org/api/worker_threads.html

Clonage structuré vs Transferable Objects

Par défaut, `postMessage(data)` copie la donnée via l'algorithme du **Structured Clone**. Le coût est proportionnel à la taille : sur plusieurs centaines de Mo, la copie devient une Long Task qui fige le thread émetteur. À mesurer soi-même dans l'onglet Performance, les chiffres dépendent de la machine et de la forme de l'objet.

Pour un transfert instantané, on transfère l'**ownership** (propriété) d'un objet transférable (`ArrayBuffer` , `MessagePort` , `ImageBitmap`) :

```
const buffer = new ArrayBuffer(1024 * 1024 * 64); // 64 Mo

// En passant buffer dans le second argument, il est transféré sans copie
worker.postMessage({ buffer }, [buffer]);

console.log(buffer.byteLength); // 0 ! La mémoire a été transférée au worker.
```

Clonage structuré vs Transferable Objects (suite)

Le transfert n'est pas à sens unique. Une fois son calcul terminé, le worker peut rendre l'ownership au thread appelant en appliquant exactement le même mécanisme, dans l'autre sens :

```
// compute.worker.js
self.onmessage = (event) => {
  const buffer = event.data.buffer;
  const view = new Uint8Array(buffer);
  // ... traitement en place sur la mémoire reçue ...

  // Le worker transfère le buffer au thread principal, et perd l'accès à son tour
  self.postMessage({ buffer }, [buffer]);
};
```

Côté `main.js`, `event.data.buffer` redevient utilisable (`byteLength` vaut de nouveau 67108864), toujours sans copie.

Clonage structuré vs Transferable Objects (fin)

À aucun moment le buffer n'appartient aux deux threads simultanément. Il n'existe qu'une seule copie physique en mémoire, et l'ownership bascule d'un thread à l'autre à chaque `postMessage` qui l'inclut dans sa liste de transfert.

Un thread qui tente d'utiliser un buffer déjà transféré tombe sur un objet **détaché** : `byteLength` vaut `0` et toute tentative de lecture échoue bruyamment, plutôt que de renvoyer des données obsolètes.

Doc MDN (Transferable Objects) : https://developer.mozilla.org/fr/docs/Web/API/Web_Workers_API/Transferable_objects

Mémoire partagée et Atomics (`SharedArrayBuffer`)

Dernier cas de figure : un `SharedArrayBuffer` n'est ni copié ni transféré, chaque thread reçoit une référence vers la **même** zone mémoire et peut l'écrire en même temps que les autres. Cet accès concurrent ouvre la porte aux *race conditions* : deux threads peuvent écraser mutuellement leur mise à jour.

D'où l'objet global `Atomics`, qui fournit des opérations indivisibles (`Atomics.add`, `Atomics.load`) ainsi qu'une synchronisation par blocage (`Atomics.wait` / `Atomics.notify`, réservés aux workers).

Doc MDN (SharedArrayBuffer) : https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer
Doc MDN (Atomics) : https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Atomics

III. Communication Réseau : Sockets & Temps Réel

Panorama des architectures de transport web

TECHNOLOGIE	PROTOCOLE	DIRECTION	FORMAT TRANSPORT	CAS D'USAGE CIBLE
Polling (Court / Long)	HTTP/1.1 ou HTTP/2	Pull périodique	Texte JSON / En-têtes complets	Fallback legacy
Server-Sent Events (SSE)	HTTP/1.1 ou HTTP/2	Push unidirectionnel (Serveur -> Client)	Flux de texte `text/event-stream`	Notifications, streaming de tokens IA
WebSocket	TCP (`ws://`, `wss://`)	Bidirectionnel Full-Duplex	Frames binaires / texte délimitées	Chat, jeux multi, collaboration temps réel
WebRTC / WebTransport	UDP (DTLS/SRTP / QUIC)	P2P ou Client-Serveur	Datagrammes non ordonnés / flux	Vidéo, visio-conférence, télémétrie rapide

Panorama des architectures de transport web (suite)

Le SSE fonctionne sur HTTP ordinaire, mais en HTTP/1.1 chaque flux occupe une des ~6 connexions autorisées par domaine. En pratique on le déploie sur HTTP/2, où le multiplexage lève cette limite.

Doc MDN (Server-sent events) : https://developer.mozilla.org/fr/docs/Web/API/Server-sent_events
Doc MDN (WebSockets API) : https://developer.mozilla.org/fr/docs/Web/API/WebSockets_API

Sockets système (TCP/UDP) vs WebSockets applicatifs

- **Socket TCP bas niveau** (ex : module `net` sous Node.js ou sockets C) : flux continu d'octets (*stream* brut sans délimitation de messages). Les navigateurs en interdisent l'accès direct pour des raisons de sécurité du modèle d'isolation web.
- **WebSocket (RFC 6455)** : protocole standardisé de couche application au-dessus de TCP, encapsulé dans des **frames** avec masquage des données côté client (*frame masking*) et support natif de formats textuels (UTF-8) et binaires.

Spécification IETF RFC 6455 : https://datatracker.ietf.org/doc/html/rfc6455

Doc Node.js (Module Net TCP) : https://nodejs.org/api/net.html

Le Handshake WebSocket (HTTP Upgrade)

Toute session WebSocket débute par un échange de mise à niveau (*Upgrade*) HTTP/1.1 :

```
GET /realtime HTTP/1.1
Host: api.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

Le Handshake WebSocket (HTTP Upgrade) (suite)

Le serveur répond avec un code de confirmation de protocole `101 Switching Protocols` :

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

À cet instant, la connexion bascule en tunnel TCP permanent sans surcoût d'en-têtes HTTP par trame.

Doc MDN (Writing WebSocket client applications) :
https://developer.mozilla.org/fr/docs/Web/API/WebSockets_API/Writing_WebSocket_client_applications

Client WebSocket résilient

En conditions réelles (perte de signal 4G, sortie de veille), la connexion peut s'interrompre. Un client professionnel reconnecte avec *jitter*.

```
class ResilientSocket {
  private socket: WebSocket | null = null;
  private attempts = 0;

  constructor(private readonly endpoint: string) {
    this.connect();
  }
```

Client WebSocket résilient (connexion)

```
private connect() {
  this.socket = new WebSocket(this.endpoint);
  this.socket.onopen = () => {
    this.attempts = 0;
  };
  this.socket.onclose = () => {
    this.retry();
  };
}
```

Client WebSocket résilient (reconnexion)

La méthode `retry` de la même classe :

```
  private retry() {
    // Exponential backoff capped, plus random jitter
    const baseDelay = Math.min(10000, 2 ** this.attempts * 1000);
    const jitter = baseDelay * 0.2 * Math.random();
    this.attempts++;
    setTimeout(() => this.connect(), baseDelay + jitter);
  }
}
```

Client WebSocket résilient (fin)

Le *jitter* a une vraie utilité : sans lui, tous les clients déconnectés par la même coupure serveur se reconnectent exactement au même instant et le resaturent aussitôt (*thundering herd*).

Doc AWS Architecture (Exponential Backoff And Jitter) : https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/

IV. Le Socle TypeScript Strict & Modélisation Fondamentale

Le mode strict : détecter les anomalies avant le runtime

Activer `"strict": true` dans `tsconfig.json` est le prérequis non négociable de toute application professionnelle. Ce paramètre regroupe un ensemble de vérifications strictes :

- `noImplicitAny` : une erreur est levée dès qu'un paramètre ou une variable retombe implicitement sur `any`.
- `strictFunctionTypes` : contrôle des paramètres de fonctions (hors syntaxe de méthode, voir section IX).

Doc officielle TypeScript (Strict Option) : https://www.typescriptlang.org/tsconfig/#strict

Bannir `any` au profit de `unknown`

- `any` désactive complètement la vérification statique du compilateur : c'est un trou noir dans le système de types.
- `unknown` représente une valeur non encore vérifiée. TypeScript refuse tout appel de méthode ou accès à une propriété tant qu'une validation préalable n'a pas été exécutée :

```
function parseId(raw: unknown): string {
  if (typeof raw !== "string") {
    throw new Error("Identifiant invalide");
  }
  return raw; // Ici, raw est garanti de type string
}
```

Bannir `any` au profit de `unknown` (suite)

Toute donnée venue de l'extérieur (réponse HTTP, `localStorage`, `JSON.parse`) entre en `unknown` et n'en sort qu'après validation. Ce principe est le point d'appui de la modélisation des données du fil rouge.

Doc officielle TypeScript (Everyday Types - unknown) : https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#unknown

Les Generics : capturer les relations de types

Un type générique paramètre un type ou une fonction pour préserver le lien entre les entrées et les sorties, sans figer la forme de la donnée :

```
function firstOrNull<T>(items: T[]): T | null {
  return items.at(0) ?? null; // `at()` renvoie T | undefined, correct même avec noUncheckedIndexedAccess
}

const firstNum = firstOrNull([10, 20, 30]); // T est inféré comme number
const firstStr = firstOrNull(["a", "b"]); // T est inféré comme string
```

Le generic évite les doublons de code tout en garantissant la cohérence des flux de données.

Doc officielle TypeScript (Generics) : https://www.typescriptlang.org/docs/handbook/2/generics.html

Utility Types indispensables (`Omit` , `Pick` , `Partial` , `Record`)

TypeScript fournit des transformations de types prêtes à l'emploi pour dériver des structures existantes :

- `Pick<T, Keys>` : extrait uniquement certaines clés de `T` .
- `Omit<T, Keys>` : exclut certaines clés de `T` (idéal pour un payload d'insertion sans `id`).
- `Partial<T>` : rend l'ensemble des propriétés optionnelles (idéal pour un patch de mise à jour).
- `Record<Keys, Type>` : construit un dictionnaire typé clé/valeur.

Utility Types indispensables (suite)

```ts
interface Article {
  id: string;
  title: string;
  content: string;
  publishedAt: string;
}

type ArticleSummary = Pick<Article, "id" | "title">;
type ArticleInput = Omit<Article, "id">;
type ArticlePatch = Partial<ArticleInput>;
```

Ces utility types servent directement à dériver les types de création et de mise à jour à partir des DTO de l'API Supabase, sans jamais redéclarer une forme à la main. Doc officielle TypeScript (Utility Types) : https://www.typescriptlang.org/docs/handbook/utility-types.html

Unions Discriminées : une union non affinée

Un article est soit un brouillon, soit publié. Ces deux états portent des données différentes. Un brouillon n'a pas de date de publication, un article publié en a forcément une.

```
type DraftArticle = { status: "draft"; publishedAt: null };
type PublishedArticle = { status: "published"; publishedAt: string };

type ArticleState = DraftArticle | PublishedArticle;
```

Face à une valeur de type `ArticleState`, le compilateur ne sait pas à laquelle des deux formes il a affaire. Le type de `publishedAt` vaut `string | null`.

Unions Discriminées : une union non affinée (suite)

```
function printYear(article: ArticleState) {
  article.publishedAt.slice(0, 4);
  // Erreur : 'article.publishedAt' is possibly 'null'
}
```

Tout appel de méthode de `string` sur `publishedAt` est refusé, puisque le compilateur ne garantit que ce qui est vrai **dans les deux cas** de l'union.

Le discriminant

Le **discriminant** est une propriété que **tous** les membres de l'union possèdent, typée avec un **type littéral** (une valeur unique, pas `string`), et dont la valeur diffère **dans chaque membre**. Ici, cette propriété est `status`, qui vaut `"draft"` d'un côté et `"published"` de l'autre. Elle joue le rôle d'une étiquette apposée sur chaque forme, lisible à l'exécution.

Ces trois conditions autorisent le compilateur à raisonner. Si `status` vaut `"published"`, alors il ne peut s'agir que de `PublishedArticle`, puisque aucun autre membre de l'union ne porte cette valeur. Ce raisonnement s'appelle l'**affinage de type** (*narrowing*).

Le discriminant (suite)

TypeScript conduit seul l'affinage dès qu'on teste le discriminant :

```
function printYear(article: ArticleState) {
  if (article.status === "published") {
    // Dans cette branche, article est de type PublishedArticle
    console.log(article.publishedAt.slice(0, 4)); // OK : publishedAt est un string
  } else {
    // Et ici, article est forcément DraftArticle
    console.log("Brouillon");
  }
}
```

Tout repose sur le littéral `status: "draft"`. Avec `status: string`, aucun affinage n'est possible, puisque la valeur `"published"` ne prouverait plus rien.

Le prédicat de type `is`

L'affinage automatique s'arrête à la frontière de la fonction. Si on extrait le test :

```
function isPublished(article: ArticleState): boolean {
  return article.status === "published";
}

function printYear(article: ArticleState) {
  if (isPublished(article)) {
    article.publishedAt.slice(0, 4); // Erreur : toujours 'possibly null'
  }
}
```

Le compilateur voit seulement qu'une fonction a renvoyé `true`. Il ignore **ce que** ce `true` signifiait. Le lien entre le résultat et le type de `article` est perdu.

Le prédicat de type `is` (suite)

Le **prédicat de type** rétablit ce lien. On remplace le type de retour `boolean` par `paramètre is Type` :

```
function isPublished(article: ArticleState): article is PublishedArticle {
  return article.status === "published";
}
```

Cette signature se lit « cette fonction renvoie un booléen, et **quand il vaut `true`, alors l'argument passé en `article` est un `PublishedArticle`** ». À l'exécution, la fonction renvoie toujours un simple booléen, `is` n'existe qu'à la compilation.

Le prédicat de type `is` (fin)

Il ajoute l'autorisation d'affiner le type de la variable passée en argument, dans la branche `if` :

```
function printYear(article: ArticleState) {
  if (isPublished(article)) {
    console.log(article.publishedAt.slice(0, 4)); // OK : article est PublishedArticle
  }
}
```

Le compilateur **ne vérifie pas** que le corps de la fonction teste réellement ce que le prédicat annonce. Un `isPublished` qui se contenterait de `return true` compile sans broncher et ment à tout le code appelant. Un prédicat reste une promesse faite au compilateur, au même titre qu'un `as`. Sa justesse relève de la responsabilité de l'auteur.

Quand utiliser le prédicat de type

Un simple `if (article.status === "published")` suffit dans la plupart des cas. Le prédicat n'a d'intérêt que si le test doit être réutilisé, notamment comme argument d'une méthode qui, elle, ne peut pas faire d'affinage automatique :

```
const articles: ArticleState[] = [];

const withoutPredicate = articles.filter((a) => a.status === "published");
// ArticleState[] : filter ne sait pas que le tableau a été purgé des brouillons

const published = articles.filter(isPublished);
// PublishedArticle[] : la surcharge de filter exploite le prédicat
```

Quand utiliser le prédicat de type (fin)

Cette modélisation rend l'état invalide **inexprimable**. Un article `draft` ne peut pas porter de `publishedAt`, et le compilateur refuse d'y accéder sans avoir d'abord vérifié le discriminant. L'erreur est arrêtée à la compilation, pas découverte en production.

Doc officielle TypeScript (Discriminated Unions) : https://www.typescriptlang.org/docs/handbook/2/narrowing.html#discriminated-unions
Doc officielle TypeScript (Type Predicates) : https://www.typescriptlang.org/docs/handbook/2/narrowing.html#using-type-predicates

Le principe KISS appliqué au typage

Un piège fréquent consiste à sur-abstraire le typage avec des génériques complexes pour des cas d'usage uniques :

```
// Approche KISS : claire, lisible et maintenable immédiatement
type SimpleArticle = {
  id: string;
  status: "draft" | "published";
  title: string;
};
```

Règle d'or : on ne généralise un type que lorsqu'il y a au moins deux cas d'usage concrets différents.

V. Template Literal Types & Mapped Types Avancés

Modificateurs de propriétés (`-readonly` , `-?`)

Les préfixes `+` et `-` ajoutent ou révoquent des contraintes de mutabilité ou d'optionalité :

Modificateurs de propriétés (`-readonly` , `-?`) (suite)

```ts
interface SchemaConfig {
  readonly host?: string;
  readonly port?: number;
}

// Rend toutes les propriétés obligatoires et modifiables
type MutableRequired<T> = {
  -readonly [K in keyof T]-?: T[K];
};

type LiveConfig = MutableRequired<SchemaConfig>;
// { host: string; port: number }
```

Doc officielle TypeScript (Mapping Modifiers) : https://www.typescriptlang.org/docs/handbook/2/mapped-types.html#mapping-modifiers

VI. Typage Nominal & Branded Types

Deux façons de décider si deux types sont compatibles

Un langage à **typage nominal** (Java, C#, Rust) compare les types par leur **nom**. Deux classes déclarées séparément restent différentes, même si elles contiennent exactement les mêmes champs. La déclaration fait l'identité.

TypeScript utilise un **typage structurel** (*structural typing*) et compare les types par leur **forme**. Une valeur ayant la forme attendue est acceptée, peu importe d'où elle vient et sous quel nom elle a été déclarée.

```
type Point = { x: number; y: number };
type Vector = { x: number; y: number };

const v: Vector = { x: 1, y: 2 };
const p: Point = v; // Accepté : même forme, donc types compatibles
```

Deux façons de décider si deux types sont compatibles (suite)

Ce choix évite de faire hériter chaque objet d'une classe commune pour décrire sa forme.

Un alias de type n'est pas un nouveau type

Le mot-clé `type` ne **crée** pas un type. Il donne un **surnom** à un type existant :

```
type UserId = string;
type PostId = string;
```

Pour le compilateur, ces deux lignes disent « `UserId`, c'est une autre façon d'écrire `string` » et « `PostId`, c'est une autre façon d'écrire `string` ». Les deux surnoms désignent donc la même chose, et sont parfaitement interchangeables :

```
function deleteUser(id: UserId) {
  /* ... */
}

const currentPostId: PostId = "post_999";
deleteUser(currentPostId); // Valide pour TypeScript ! Pourtant c'est une inversion d'identifiant fatale.
```

Un alias de type n'est pas un nouveau type (suite)

Le compilateur ne voit ici que `deleteUser(string)` appelée avec un `string`. Il n'a aucun moyen de savoir que la sémantique diffère. L'erreur ne sera découverte qu'en production, avec le mauvais enregistrement supprimé.

On appelle **primitive obsession** cette habitude de représenter des concepts métier distincts (identifiant utilisateur, identifiant article, email, montant en centimes) par le même type primitif. Le typage structurel la rend invisible au compilateur.

Doc officielle TypeScript (FAQ: Nominal and Structural Typing) : https://github.com/microsoft/TypeScript/wiki/FAQ#can-i-make-a-type-alias-nominal-or-opaque

Solution : les Branded Types, rendre les formes différentes

Puisque TypeScript compare les formes, la seule façon de séparer deux `string` consiste à leur donner des formes différentes. On ajoute donc à chacun une propriété **fictive**, une marque (*brand*), qui n'existe que dans le système de types :

```
type UserId = string & { readonly brand: "UserId" };
type PostId = string & { readonly brand: "PostId" };
```

L'opérateur `&` forme un **type intersection**, qui se lit « un `string` **et en plus** un objet portant cette propriété `brand` ». Les deux formes ne sont plus identiques, donc le compilateur refuse de les confondre.

Solution : les Branded Types (suite)

Aucune valeur ne portera jamais réellement cette propriété à l'exécution. Elle sert uniquement d'étiquette au compilateur, ce qui vaut à ce pattern le nom de *phantom type*.

La version robuste, morceau par morceau :

```
declare const __brand: unique symbol;

type Brand<T, B extends string> = T & { readonly [__brand]: B };

type UserId = Brand<string, "UserId">;
type PostId = Brand<string, "PostId">;
```

Solution : les Branded Types, détail de la version robuste

- `declare const __brand` : `declare` annonce au compilateur l'existence d'une variable **sans générer la moindre ligne de JavaScript**. Le symbole n'existe qu'à la compilation.
- `unique symbol` : un type de clé garanti unique. Utiliser une clé symbole plutôt que `brand: string` évite toute collision avec une propriété réelle et empêche qu'on puisse écrire la marque à la main par accident.
- `Brand<T, B>` : un générique réutilisable. `T` est le type réel transporté (`string` , `number` ...), `B` est l'étiquette qui distingue les marques entre elles.

Le constructeur, seul endroit où l'on triche

```ts
// Le `as` est confiné dans un unique constructeur, qui est aussi l'endroit où valider la donnée.
function toUserId(raw: string): UserId {
  if (!raw.startsWith("usr_"))
    throw new Error("Identifiant utilisateur invalide");
  return raw as UserId;
}

function deleteUser(id: UserId) {
  /* ... */
}

const user = toUserId("usr_123");
const post = "pst_456" as PostId;

deleteUser(user); // Valide
// deleteUser(post)
// Type 'Brand<string, "PostId">' is not assignable to type 'Brand<string, "UserId">'
```

Le constructeur, seul endroit où l'on triche (suite)

Le compilateur ne créera jamais de valeur portant la marque. C'est au développeur de la lui affirmer, avec un `as`. La discipline consiste à confiner ce `as` dans **une unique fonction**, qui est aussi l'endroit naturel où valider la donnée.

Ce confinement fait tout l'intérêt du pattern. Il existe désormais **un seul point d'entrée** pour fabriquer un `UserId`, donc un seul endroit à auditer. Partout ailleurs dans le code, un `UserId` est garanti validé. Contourner la règle en écrivant `"n'importe quoi" as UserId` reste possible. Le compilateur ne peut pas imposer cette convention d'équipe, qui se surveille en revue de code ou via une règle ESLint sur les assertions.

Coût et périmètre des Branded Types

La marque n'existe qu'à la compilation. À l'exécution, un `UserId` reste une chaîne ordinaire, sans propriété supplémentaire ni coût mémoire. `console.log(user)` affiche `"usr_123"`, et la valeur part telle quelle dans une requête HTTP.

Le gain est réel, le prix aussi (un constructeur par identifiant, et des conversions explicites aux frontières). Réserver les branded types aux valeurs qu'on risque vraiment d'intervertir, typiquement les identifiants de deux tables manipulés dans la même fonction.

VII. L'Opérateur `satisfies` & Notions de Variance

Pourquoi `satisfies` surpasse l'annotation classique

L'opérateur `satisfies` (TS 4.9+) valide qu'une expression respecte un type cible **sans écraser le type inféré exact**.

Annoter, c'est imposer un type. Une annotation `const x: T = ...` vérifie que la valeur est compatible avec `T`, et surtout elle **déclare que la variable est de type `T`**. Le type réel de la valeur écrite à droite est oublié. Toute lecture ultérieure passe par `T`.

```
type Color = string | [r: number, g: number, b: number];
type Palette = Record<string, Color>;

const paletteA: Palette = { primary: "#2563eb" };
```

`satisfies` : le prix de l'annotation classique

`paletteA` est de type `Palette`, donc `Record<string, Color>`. Le compilateur sait seulement que `paletteA.primary` est un `Color`, et `Color` accepte aussi le tuple `[number, number, number]`.

```
paletteA.primary.toUpperCase();
// Property 'toUpperCase' does not exist on type 'Color'.
// Property 'toUpperCase' does not exist on type '[r: number, g: number, b: number]'.
```

Le refus n'est pas une erreur du compilateur. Il applique ce qu'on lui a demandé de croire. La méthode `toUpperCase` n'existant pas sur les tableaux, elle est rejetée.

`satisfies`, vérifier sans imposer

`satisfies` sépare les deux rôles. Il effectue la vérification de compatibilité, puis laisse la variable garder le type que TypeScript a **inféré** de la valeur écrite.

```
const paletteB = {
  primary: "#2563eb",
} satisfies Palette;
```

Le type de `paletteB` est `{ primary: string }`, pas `Palette`. La valeur écrite est une chaîne, donc le type retenu est `string`, et `Color` n'apparaît jamais.

```
paletteB.primary.toUpperCase(); // Valide et auto-complété
```

`satisfies`, la vérification a bien eu lieu

Une valeur hors contrat est refusée à la ligne du `satisfies` :

```
const paletteC = {
  primary: 42,
} satisfies Palette;
// Type 'number' is not assignable to type 'Color'.
```

`satisfies`, ce que l'inférence conserve en plus

Le bénéfice ne se limite pas aux valeurs. Les **clés** exactes sont conservées elles aussi, alors que `Record<string, Color>` les efface au profit de `string`.

```
const themeA: Palette = { primary: "#2563eb", danger: "#dc2626" };
themeA.typoNonExistante; // Aucune erreur, Record<string, Color> accepte n'importe quelle clé

const themeB = {
  primary: "#2563eb",
  danger: "#dc2626",
} satisfies Palette;

themeB.typoNonExistante;
// Property 'typoNonExistante' does not exist on type '{ primary: string; danger: string; }'

type ThemeKey = keyof typeof themeB; // "primary" | "danger"
```

`satisfies`, ce que l'inférence conserve en plus (fin)

`satisfies` convient aux objets de configuration, aux dictionnaires de routes et aux états initiaux, où l'on veut à la fois le contrôle d'un schéma et l'autocomplétion sur les clés réellement présentes.

Doc officielle TypeScript (The satisfies Operator) : https://www.typescriptlang.org/docs/handbook/release-notes/typescript-4-9.html#the-satisfies-operator

`satisfies` vs Type Assertion (`as`)

- `as T` : force la main du compilateur et masque de potentielles incohérences d'implémentation.
- `satisfies T` : valide l'adéquation au contrat d'interface tout en conservant le typage au plus précis.

Règle : réserver `as` aux cas où l'on dispose d'une information que le compilateur ne peut pas avoir.

Principes de Variance : deux notions préalables

Un type `A` est un **sous-type** de `B` quand toute valeur de `A` est utilisable partout où une valeur de `B` est attendue.

```
type User = { id: string; name: string };
type Admin = { id: string; name: string; permissions: string[] };
```

`Admin` est un sous-type de `User`, puisqu'un `Admin` possède tout ce que `User` exige, et davantage. Une fonction qui réclame un `User` se contente donc très bien d'un `Admin`.

Un **type construit** est un type paramétré par un autre. `User[]`, `Promise<User>`, `() => User` et `(u: User) => void` sont quatre types construits à partir de `User`.

Covariance sur les valeurs de retour

Une fonction qui promet de renvoyer un `User` peut être remplacée par une fonction qui renvoie un `Admin`. Le sous-typage se propage dans le même sens : c'est la **covariance**.

```
type GetUser = () => User;

const getAdmin = (): Admin => ({ id: "1", name: "Ada", permissions: ["all"] });

const get: GetUser = getAdmin; // Accepté
const displayed = get(); // Typé User, et c'est bien un objet complet

// L'inverse est refusé
type GetAdmin = () => Admin;
const getUser = (): User => ({ id: "2", name: "Bob" });
const broken: GetAdmin = getUser;
// Property 'permissions' is missing in type 'User' but required in type 'Admin'
```

Covariance sur les valeurs de retour (suite)

L'appelant de `GetUser` attend un objet avec `id` et `name`. En recevoir un qui porte en plus `permissions` ne casse rien. À l'inverse, un appelant de `GetAdmin` lirait `permissions` sur un objet qui n'en a pas.

Contravariance sur les paramètres

Sur les paramètres, la relation s'inverse. Une fonction acceptant un `User` peut remplacer une fonction censée accepter un `Admin`, jamais le contraire. C'est la **contravariance**, active sous `strictFunctionTypes`.

```
type AdminHandler = (admin: Admin) => void;

const logAnyUser = (user: User) => console.log(user.name);

const handle: AdminHandler = logAnyUser; // Accepté
```

`logAnyUser` se contente de lire `name`. Comme tout `Admin` possède un `name`, elle traite sans risque tout ce que `handle` lui enverra.

Contravariance sur les paramètres (suite)

Le sens inverse produit une erreur d'exécution, et TypeScript le refuse pour cette raison :

```
type UserHandler = (user: User) => void;

const logPermissions = (admin: Admin) => console.log(admin.permissions.length);

const callback: UserHandler = logPermissions;
// Types of parameters 'admin' and 'user' are incompatible.
// Property 'permissions' is missing in type 'User'

callback({ id: "3", name: "Chris" }); // Aurait planté : permissions est undefined
```

Le cas des tableaux, une exception assumée

`Array<T>` utilise son paramètre dans les deux positions à la fois. On lit un tableau, ce qui est une position de sortie, et on y écrit, ce qui est une position d'entrée. La sûreté exigerait donc qu'il soit **invariant**. TypeScript le traite pourtant comme covariant.

```
const admins: Admin[] = [{ id: "1", name: "Ada", permissions: ["all"] }];
const users: User[] = admins; // Accepté

users.push({ id: "4", name: "Dan" }); // Accepté aussi

admins[1].permissions.length;
// Erreur d'exécution : Cannot read properties of undefined
```

Le cas des tableaux, une exception assumée (suite)

Le compilateur laisse passer les deux lignes et le programme casse à l'exécution. L'équipe TypeScript a retenu ce compromis pour garder le langage utilisable sur le code JavaScript existant.

La conséquence pratique : ne jamais laisser une fonction écrire dans un tableau qu'elle a reçu élargi. Un paramètre `readonly User[]` supprime le problème en interdisant l'écriture.

Où la variance se rencontre vraiment

Les erreurs de variance apparaissent surtout sur les callbacks et les reducers, là où une fonction est passée en valeur à une API qui la rappellera plus tard. Un `onChange` typé trop étroitement, un `reducer` dont le paramètre d'action est plus spécifique que l'union déclarée, et le compilateur refuse l'affectation.

La règle à appliquer est d'élargir les paramètres et de restreindre les retours.

Doc officielle TypeScript (strictFunctionTypes) : https://www.typescriptlang.org/tsconfig/#strictFunctionTypes